

Miniteste M4 — Conversão analógico-digital

Microcontroladores · DENE/UFMT · GABARITO

Nome: _____

20 minutos

Todos os dados de que você precisa estão na caixa abaixo — nenhuma questão se responde procurando. As respostas valem pelo raciocínio; um número certo sem justificativa vale metade.

DADOS

Referência de fundo de escala: $V_{DD} = 5\text{ V}$

Resolução do conversor: 10 bits

Sensor LM35: 10 mV por $^{\circ}\text{C}$

Aquisição: $4 T_{AD}$ Conversão: $11 T_{AD}$

$T_{AD} = 1\text{ }\mu\text{s}$ ($ADCS = 101$, $F_{osc}/16$)

Ciclo de máquina a 16 MHz: 250 ns

Questão 1 (2,0)

Um estudante liga um botão em RB0 e escreve, como primeira linha do `main`:

```
while (1) { if (PORTBbits.RB0 == 0) LATD = 0xFF; else LATD = 0x00; }
```

Com a chave pressionada ou solta, `PORTBbits.RB0` devolve sempre 0 — e os LEDs ficam sempre acesos. O botão foi testado no multímetro e está bom.

(a) O que está acontecendo com o pino? Nomeie o mecanismo.

(b) Um colega sugere acrescentar `#pragma config PBDEN = OFF` no início do arquivo. Avalie a sugestão e diga o que resolve.

RESPOSTA

(a) RB0 nasceu analógico. O bit de configuração `PBDEN` vale 1 de fábrica, e com ele RB0 a RB4 saem do reset como entradas analógicas. Num pino em modo analógico o buffer de entrada digital está desligado, e a leitura de `PORT` devolve 0 qualquer que seja a tensão no pino. O defeito não está no botão nem no `if` — está no caminho de entrada que o reset escolheu.

(b) Não resolve. As palavras de configuração pertencem ao bootloader — é o código que veio antes do seu, do encontro 0 — e o `#pragma config` da aplicação é ignorado. `PBDEN` não é ajustável pelo programa do aluno.

O que resolve é desfazer o efeito em tempo de execução, antes de qualquer leitura:

```
ADCON1 = 0x0F; /* PCFG = 1111: todos os canais digitais */
```

Questão 2 (2,0)

Um estudante monta um divisor com dois resistores de $1\text{ M}\Omega$ entre 5 V e o terra, e liga o ponto do meio em AN0. O multímetro no pino mede 2,50 V, como esperado. O conversor devolve um código perto de 140, e não de 512.

(a) Explique o defeito a partir do que o conversor realmente mede.

(b) O estudante passa a ler dezesseis vezes e tirar a média. O código melhora? Justifique, e diga o que de fato corrige.

RESPOSTA

(a) O conversor não lê o pino: lê um capacitor de retenção, de cerca de 25 pF, que precisa se carregar através da resistência da fonte durante o tempo de aquisição. A resistência equivalente do divisor é $1\text{ M}\Omega$ em paralelo com $1\text{ M}\Omega$, ou 500 k Ω , e a constante de tempo fica em $500\text{ k}\Omega \times 25\text{ pF} = 12,5\text{ }\mu\text{s}$.

A aquisição dura $4 T_{AD} = 4\text{ }\mu\text{s}$, menos de um terço de uma constante de tempo. O capacitor chega a cerca de 27% dos 2,50 V, ou 0,69 V — que em dez bits dá justamente algo perto de 140. O multímetro não vê o problema porque a entrada dele tem dezenas de megaohms e praticamente não puxa carga; o ADC puxa.

(b) Não melhora nada. O erro é **sistemático**: todas as dezesseis amostras têm o mesmo déficit de carga, e a média de dezesseis valores igualmente errados é o mesmo valor errado. Média reduz variância, não viés.

O que corrige é diminuir a impedância da fonte — resistores muito menores no divisor — ou pôr um seguidor de tensão entre o divisor e o pino. Alternativa válida: aumentar **ACQT**, mas para 12,5 μs de constante de tempo o tempo de aquisição necessário fica caro.

Questão 3 (2,0)

- (a) Quanto vale um degrau do conversor, em milivolts e em graus Celsius com o LM35? Mostre as duas contas.
- (b) Um colega propõe ligar uma referência estável de 1,00 V no pino RA3 e usá-la como VREF+, “para ganhar resolução”. Quanto ele ganha? Cite duas coisas que ele perde.

RESPOSTA

(a) $5\text{ V} \div 1024 = 4,8828125\text{ mV}$. Não é arredondamento: 1024 é potência de dois e a divisão é exata em binário.

Com o LM35 entregando 10 mV por grau, um degrau vale $4,8828125 \div 10 = 0,48828125\text{ }^\circ\text{C}$ — cerca de meio grau.

Nenhuma leitura desta cadeia distingue 24,7 de 25,0 $^\circ\text{C}$, e isso não melhora com código melhor.

(b) Ganha cinco vezes: $1\text{ V} \div 1024 = 0,977\text{ mV}$, ou 0,098 $^\circ\text{C}$ por degrau.

Perde, entre outras coisas:

— **um canal**: RA3 deixa de estar disponível como AN3, porque virou a entrada de referência;

— **a estabilidade vira erro de medida**: a referência passa a ser um componente do qual toda leitura depende, e a deriva dela com temperatura e tempo entra diretamente no resultado. Com VDD o problema existe também, mas a referência é a mesma que alimenta o sensor;

— **a faixa**: o fundo de escala cai para 1 V, ou 100 $^\circ\text{C}$ de LM35. Suficiente para o termostato, mas satura acima disso.

Aceitar dois dos três.

Questão 4 (2,0)

A conversão de código para décimos de grau é exata em inteiros, porque $4,8828125 = 625/128$. Um estudante escreve:

```
uint16_t n = ((uint16_t) ADRESH << 8) | ADRESL; /* 0 a 1023 */
uint16_t decimos = (n * 625u) >> 7;
```

- (a) Com $n = 1023$, que valor aparece em **decimos**, e que valor deveria aparecer? Mostre a conta.
- (b) Por que esse defeito é mais perigoso que um que trava o programa?

RESPOSTA

(a) A conta correta é $1023 \times 625 = 639375$, e $639375 \div 128 = 4995$ décimos, ou 499,5 $^\circ\text{C}$ — o fundo de escala, como esperado.

O que acontece: os dois operandos são de 16 bits, então o produto é calculado em 16 bits e 639375 não cabe. Sobra o resto da divisão por 65536:

$$639375 - 9 \times 65536 = 49551 \quad \text{e} \quad 49551 \div 128 = 387$$

Aparece 387, isto é **38,7 $^\circ\text{C}$** . A correção é forçar a conta em 32 bits: `((uint32_t) n * 625u) >> 7`.

(b) Porque 38,7 $^\circ\text{C}$ é um número **plausível**. Nada trava, nada avisa, nenhum valor absurdo aparece na tela — e o termostato vai agir sobre ele. Um programa que trava é um programa que denuncia o próprio defeito; este mente com naturalidade, e só é descoberto por quem conferir a conta ou aquecer o sensor até o fim da escala.

Questão 5 (2,0)

- (a) Quanto custa uma leitura completa do conversor, em microssegundos e em ciclos de máquina? Uma atualização das duas linhas do display custa cerca de 5200 ciclos. Que consequência de projeto sai da comparação?
- (b) Um estudante lê o LM35 quatro vezes seguidas e divide por quatro, esperando “ganhar resolução”. Com o sensor parado, as quatro leituras devolvem exatamente o mesmo código. Explique por que a média não ajudou, e diga sob que condição ela ajudaria.

RESPOSTA

(a) $4 T_{\text{AD}}$ de aquisição mais $11 T_{\text{AD}}$ de conversão dão $15 T_{\text{AD}}$. A $T_{\text{AD}} = 1\text{ }\mu\text{s}$ são 15 μs , e $15\text{ }\mu\text{s} \div 250\text{ ns} = \mathbf{60\text{ ciclos de máquina}}$.

A comparação: 60 contra 5200. Medir custa cerca de 1,2% do que custa mostrar.

A consequência é contraintuitiva: **num termostato o gargalo não é medir, é escrever**. Quem precisar de tempo de processador não deve amostrar menos — deve atualizar a tela menos.

(b) Porque a média só recupera informação entre degraus se houver ruído **maior que um LSB** somado ao sinal. Com ruído, amostras sucessivas caem em degraus diferentes e a proporção entre elas carrega a informação intermediária. Sem ruído, as quatro amostras caem no mesmo degrau, e a média de um valor com ele mesmo é ele mesmo.

A quantização já descartou a informação; nenhuma soma a traz de volta.

A média continua útil para o que ela de fato faz: reduzir a variância de leitura quando há interferência da rede, do PWM do cooler ou do relé chaveando.